

NinaXander: Conditions and Limits of Composing Frozen Language Models Across Model Families via a Shared Latent Representation

TAKANORI KOTAMA¹ SHUN-ICHIRO HAYASHI¹ DAICHI MUKUNOKI²
TETSUYA HOSHINO² TAKAHIRO KATAGIRI²

Abstract: NinaXander names a family of composed language models that connect frozen layers from different pretraining families through a shared-latent adapter. We study the recurrent RWKV-4-Raven-7B and the Transformer-based Tulu-Pythia-6.9b. An individual composed model is specified by its cross-family path and switch layer, as in `Pythia→RWKV` ($L=4$). We test whether representations at corresponding layers can be connected by a comparatively lightweight adapter, and what effect that connection has on the model’s language performance. There are four main results. First, under the matched conditions of a shared tokenizer, width, depth, and training domain, the centered latent correlation reaches $\rho_{ctr}=0.901$. Shuffling the row correspondence destroys the alignment, yet the CKA of the raw representations does not peak at same-numbered layers, and performance degrades sharply under a domain shift to WikiText. The result is therefore not a general semantic space but a token-aligned representation that is learnable under favorable conditions. Second, the four-path read-outs are 0.695/0.559/0.711/0.590 for `RWKV self-recon/RWKV→Pythia/Pythia self-recon/Pythia→RWKV`; the method uses no compression stage, yet reconstruction error remains. Third, the best direct affine maps reach 0.487/0.535 for `RWKV→Pythia/Pythia→RWKV`, while 13.4%/12.0% of the trained adapter’s output remains unexplained by the respective best affine fits; disabling the same adapter’s nonlinear branch collapses both cross read-outs to negative R^2 . Fourth, chimeras composed at a single boundary produce syntactically plausible generations and perform multiple choice, but do not reach the stronger parent. `Pythia→RWKV` exceeds `RWKV→Pythia` at deep boundaries by 8.9–18.8 accuracy points at the same layer, and the `Pythia-prefix→RWKV-suffix` `Pythia→RWKV` ($L=4$) keeps only 5 Transformer layers, cutting KV by 84.4% while retaining ARC-Easy 62.2% and SciQ 69.2%. It does not, however, significantly exceed RWKV, and this choice of $L=4$ is itself a post-hoc selection on the same two test sets. Generalization to out-of-domain WikiText also remains a substantial problem.

1. Introduction

Pretraining a language model demands enormous computation, yet its result remains a set of weights tied to one architecture. Changing the model’s inference characteristics may require reducing a KV cache that grows with context length. The usual options are to pretrain a new hybrid that mixes recurrent and Transformer layers, or to convert or distill an existing model by replacing sequence-mixing layers such as attention with a cheaper mechanism. Both approaches train new weights inside the model. They do not recompose already-trained blocks as frozen components.

Existing approaches share this constraint. A hybrid trained from scratch pretrains the whole model, and the conversion-and-distillation line retrains the sequence-mixing weights. Work that stitches frozen models together does exist, but only with Transformers on both sides. Weight-space merging presupposes compatible weights. Recomposing fin-

ished, frozen checkpoints as parts, across families, remains open.

That is the idea this work tests. If the intermediate representations of independently trained heterogeneous models can be translated into each other, existing checkpoints become frozen reusable parts. Training one adapter makes many composition points available at once, after which the accuracy–memory tradeoff can be searched *post hoc*, without per-configuration retraining. We use four switch positions in both directions, drawing eight configurations from one adapter and computing their non-dominated set. The adapter itself still requires substantial training. Section 3.6 gives the training configuration. This study does not establish a total training-cost advantage over pretraining or distilling a new hybrid.

As a first step, we build a family of *chimera models* called NinaXander.^{*1} These models connect frozen blocks from

¹ Graduate School of Informatics, Nagoya University

² Information Technology Center, Nagoya University

^{*1} *NinaXander* derives from the chimera in *Fullmetal Alchemist*: the synthesis succeeds, but does not reach what it was made from.

different families through one trained adapter. Section 3 specifies each model by its cross-family path and switch layer. RWKV is a model family whose linear-attention computation also admits recurrent execution, while Tulu-Pythia is an instruction-tuned Transformer. We study RWKV-4-Raven-7B and Tulu-Pythia-6.9b under the favorable conditions of a shared tokenizer, width, and depth. We ask three questions. Does cross-family composition work? How do function and KV cost vary with the composition point? What limits the connection?

The result is not a competitive new model. No configuration surpasses the stronger parent. Instead, the experiments show that frozen assets can be partly recomposed after training, and that imperfect alignment, reconstruction error, and domain shift obstruct the result. The concrete gain is a configuration that retains only 5 Transformer layers, cuts KV by 84.4%, and preserves part of the multiple-choice accuracy. This work makes four contributions: alignment measurement after removal of the common mode; a controlled comparison of RWKV self-recon, RWKV→Pythia, Pythia self-recon, and Pythia→RWKV; an intervention within the same mapping that measures adapter nonlinearity on both cross-family paths; and measurements of accuracy, memory, domain generalization, and path asymmetry for single-boundary RWKV→Pythia and Pythia→RWKV chimeras.

2. Background and Related Work

2.1 The residual stream and what composition means

An autoregressive language model updates a per-position representation, the residual stream, layer by layer. In a model of width d , block ℓ has the form $h^{(\ell+1)} = h^{(\ell)} + f_\ell(h^{(\ell)})$, with f_ℓ performing sequence mixing and channel mixing. Composing at a depth boundary L means running one model's blocks 0 through L , then feeding the resulting $h^{(L+1)}$ to the other model's block $L+1$. Even when width and depth agree, $h^{(L+1)}$ lives in the coordinate system that each model's own training fixed, so handing one model's value to the other does not match the distribution the downstream weights expect. Whether the composition is possible therefore reduces to whether the residuals of the two families can be translated into each other.

2.2 The asymmetry in inference cost

The two families differ in how their inference cost grows. A layer with softmax attention must retain the keys and values of every past token, so it needs a cache of $O(T)$ in the context length T . A layer that can be executed as linear attention, or in an equivalent recurrent form, carries only a fixed-size state and is therefore $O(1)$. This asymmetry is what makes the composition worth studying here: which family sits upstream and which downstream decides how many Transformer layers remain, and with them how much cache must be kept.

2.3 The scale of the residual

Residual scale differs from layer to layer, and does not agree across families. Comparing residuals across layers, and handing them over at a boundary, therefore both require standardization by statistics estimated per layer. This is why the adapter in this work is wrapped in per-layer standardization and its inverse. Section 3 gives the exact operations.

2.4 Model stitching

Connecting the layers of 2 frozen models through a trainable stitching layer was introduced by Lenc and Vedaldi [1] to measure the *equivalence* of representations. They splice the lower layers of one frozen CNN onto the upper layers of another with a linear stitching layer and score the resulting “Franken-CNN” by the original task loss. Their experiments include pairs trained on ImageNet and MIT Places, as well as pairs drawn from genuinely different architectures such as AlexNet, VGG-16, and ResNet-50. Bansal et al. [2] revisited stitching as a probe for representation comparison. They varied the random seed, training subset, training duration, width, and objective while retaining the same architecture, and identified cross-architecture stitching as an open difficulty. Csizsárik et al. [3] use a single affine stitching layer to match the intermediate representations of two trained networks. They argue that agreement should be judged by the task performance of the stitched network rather than by a representation-similarity index alone, and show that existing similarity indices do not always track task performance. Our separation of alignment from decodability follows this view. These studies concern vision models. Our work instead joins the distinct RNN and Transformer families through a shared latent representation that translates the residual once at a layer boundary.

2.5 Composing frozen LLMs

Recent work continues to compose frozen LLMs through trainable connections. StitchLLM [4] stitches blocks of Transformers of different scales with stitching layers and selects a path at serving time, and BTS [5] exchanges hidden states *bidirectionally* between a frozen seed and frozen experts branched from that seed, through Experts-into-Hub and Hub-into-Experts stitch layers alternating every few layers. CALM [6] fuses the intermediate representations of 2 frozen Transformers via cross-attention, and Manticore [7] runs Transformer and Mamba block sequences *in parallel* behind projections, mixes them by a learned convex combination, and fine-tunes the resulting hybrid end to end. Its embeddings and language-model head are newly initialized.

A separate line of work translates activations between frozen models. Moschella et al. [8] re-encode samples relative to fixed anchors and thereby absorb the transformation between independently trained latent spaces without learning a translator. Maiorca et al. [9] estimate a map between two pretrained latent spaces in closed form from a small set of semantic correspondences. The affine control in Table 1

is a low-capacity direct-map baseline of this kind.

Model Alignment Search [10] maps activations bidirectionally between a frozen recurrent network and a frozen Transformer, but as a measure of representational similarity rather than to build a composed model. Chen et al. [11] map the residual streams of 2 frozen language models into each other bidirectionally with an affine map at a single layer, and evaluate the next-token cross-entropy of the composite that runs one model’s prefix into the other’s suffix. Their pairs are same-family Transformers sharing a tokenizer, and they identify the cross-family setting as future work.

The Bicameral Model preprint [12] couples 2 frozen language models through a bidirectional translation network, but there each model still executes its own full stack in parallel. The GoldFinch preprint [13] stacks a Transformer on top of RWKV-6 at a single boundary and compresses the KV cache by $756\text{--}2550\times$, but trains the model from scratch rather than composing pretrained parents. Oozeer et al. [14] learn activation maps between frozen LLMs of different families to transfer safety interventions and steering vectors. Their use of an affine control is close to ours, but the transfer remains at one layer and they do not run a composed model. Our affine control is fitted directly to paired residuals and therefore uses stronger supervision than zero-shot relative representations.

Prior work therefore either places Transformers on both sides, fine-tunes the composed model as Manticore does, or has at most one pretrained parent. Chen et al. [11] have already established the mechanism of a single boundary, two directions, and frozen parents for same-family Transformer pairs that share a tokenizer. We do not claim that mechanism as novel. We address the cross-family case that they left open and test whether the same connection works between a linear-attention recurrent model and a Transformer. To our knowledge, no prior study has composed these two families at a single boundary while freezing both parents and both language-model heads, then evaluated the result as a working generator. We further report the KV-cache reduction that this composition yields.

2.6 Representation similarity

Centered kernel alignment (CKA) [15] and (SV/PW)CCA [16], [17] are standard tools for measuring representational similarity between networks. Our work uses them as auxiliary metrics for the all-layer comparison while placing its main focus on *functional, reconstruction-mediated* stitching. On cross-architecture universality, Wang et al. [18] report that sparse-autoencoder features and induction circuits are largely similar between Transformers and Mamba. Their result provides prior evidence for our premise, although their recurrent family is Mamba rather than RWKV.

2.7 Model merging and knowledge fusion

Weight-space methods such as model soups [19], TIES [20], and evolutionary merging [21] presuppose

identical or compatible weights. FuseLLM [22] fuses output distributions and does not require weight compatibility, but it needs token alignment across vocabularies and continued training of the target model. Our work is complementary in that it merges no weights at all and instead translates the *activation space* to recombine frozen blocks.

3. Method

3.1 Target models

We use the instruction-following models RWKV-4-Raven-7B and Tulu-Pythia-6.9b. We denote the former by model *A* and the latter by model *B*. Both models have 32 blocks, hidden width $d=4096$, and the GPT-NeoX-20B tokenizer. The shared tokenizer turns the same input into the same token sequence, so residuals at each token position can be paired directly. Equal depths let us pair block j with block j , and equal hidden widths make the vectors at the boundary dimensionally compatible. We selected the pair deliberately because these three conditions facilitate composition. Model pairs with different tokenizers, depths, or hidden widths are outside the scope of this study.

The base models were pretrained on The Pile [23], an English corpus assembled by EleutherAI. The Pile is an 825 GiB language-modeling corpus that combines 22 data sources, including scholarly articles, web documents, books, and source code. The base corpus is shared, but the subsequent instruction-tuning data and prompt formats differ.

RWKV [24] uses the WKV operator for linear-time sequence mixing in place of softmax dot-product attention and can execute the same computation in either parallel or recurrent form. Raven is an instruction-following RWKV-4 model developed by the RWKV project and released by BlinkDL. It starts from RWKV-4-Pile-7B and receives further training on general instruction datasets such as Alpaca, Guanaco, and GPT4All, as well as Code-Alpaca for code tasks and ShareGPT for conversations. Code-Alpaca is a synthetic dataset of approximately 20,000 instructions and responses, mainly for code generation, editing, and optimization [25]. ShareGPT is a collection of multi-turn conversations with ChatGPT that users shared publicly [26]. We converted the Raven 7B weights distributed by BlinkDL to Hugging Face format and re-saved them in fp16 for this study.*² The weights we used contain 7,392,649,216 parameters, with 32 blocks, hidden width 4096, and intermediate width 16384. The converted configuration sets `context_length` to 1024. This value is defined as the maximum sequence length handled in one forward pass. The recurrent form can carry a fixed-size state whose size does not depend on context length.*³ The filename and revision of the source `.pth` checkpoint were not preserved, so we cannot identify the training context length claimed for that upstream checkpoint. Recurrent execution also does not guarantee language

*² BlinkDL, RWKV-4 Raven model repository, <https://huggingface.co/BlinkDL/rwkv-4-raven>.

*³ Hugging Face Transformers, RWKV documentation, https://huggingface.co/docs/transformers/model_doc/rwkv.

quality beyond the length used in training or specified by the configuration.

Pythia [27] is a family of GPT-NeoX Transformers developed by EleutherAI for research. Tulu-Pythia-6.9b is Pythia-6.9b fully fine-tuned by the Allen Institute for AI on the Tulu instruction mixture [28]. The mixture includes FLAN V2, Chain-of-Thought, Dolly, OpenAssistant 1, GPT4-Alpaca, Code-Alpaca, and ShareGPT.*⁴ The weights we used contain 6,856,040,448 parameters, with 32 blocks, hidden width 4096, and 32 attention heads. The configured maximum number of positions is 2048.*⁵ Based on the distributors' descriptions, Code-Alpaca and ShareGPT are the only instruction-tuning datasets named for both models. The models were therefore not trained on completely independent data, although their architectures and most of their instruction data differ.

3.2 Residual pairs and notation

In this paper, a *residual* is not an error term. It is the hidden vector passed from one block to the next. Composing the models means translating this vector from one model's coordinate system into the other model's coordinate system. For model $M \in \{A, B\}$, let $r_M^j[t] \in \mathbb{R}^{4096}$ denote the unstandardized residual after block j at token position t . The block index is $j \in \{0, \dots, 31\}$, and t indexes a token in the input. We feed the same token sequence to both models and treat $r_A^j[t]$ and $r_B^j[t]$ as one paired training example.

Residual scale differs by model and layer, so we estimate a per-dimension mean and standard deviation from the training residuals of each layer. The operation std_M^j standardizes a residual with these statistics, while unstd_M^j reverses that operation and restores the original scale. We write the standardized residual as $x_M^j[t] = \text{std}_M^j(r_M^j[t])$. For readability, the training objective below suppresses j and t and writes these vectors as x_A and x_B .

3.3 Shared-latent adapter

The adapter has one encoder and one decoder for each model. Encoders E_A, E_B map standardized residuals into a shared latent space, while decoders D_A, D_B reconstruct each model's standardized residual from that space. Subscripts A and B identify the model to which a map belongs. Vectors $z_A, z_B \in \mathbb{R}^{d_z}$ are the two latent representations obtained at the same token position, and their latent width is $d_z=4096$. Let the hyperparameter $\lambda \geq 0$ specify how strongly the latent-alignment error is weighted relative to the two self-reconstruction errors. The training objective is

$$\begin{aligned} z_A &= \text{LN}(E_A(x_A)), & z_B &= \text{LN}(E_B(x_B)), \\ \mathcal{L} &= \|D_A(z_A + \xi_A) - x_A\|^2 + \|D_B(z_B + \xi_B) - x_B\|^2 \\ &\quad + \lambda \|z_A - z_B\|^2. \end{aligned} \quad (1)$$

The first two terms of Eq. (1) are the self-reconstruction errors for models A and B . The third term is a latent-alignment error that brings z_A and z_B from the same token position closer together. Increasing λ increases the relative weight of this third term in the training loss $\mathcal{L}$. Each norm is implemented as a mean squared error over all tokens and dimensions. Here LN is LayerNorm without a trainable scale or bias. The vectors $\xi_A, \xi_B \sim \mathcal{N}(0, \sigma^2 I)$ are independent, zero-mean Gaussian noise added only before decoding; $\sigma > 0$ is the noise-standard-deviation hyperparameter, and I is the $d_z \times d_z$ identity matrix. Throughout adapter training, we fix $\lambda=1$ and $\sigma=0.4$. Setting $\lambda=1$ gives the three mean-squared-error terms the same nominal coefficient of 1; it does not imply that their measured magnitudes are equal. Neither value is tuned by layer, path, or task, and we do not ablate either value.

The loss contains no term that directly predicts a residual of B from a residual of A , or vice versa. The cross-family map sends z_A from encoder A to decoder B and reads out $D_B(z_A)$. The reverse direction reads out $D_A(z_B)$. These maps can work only if the third term brings z_A and z_B into the same coordinate system. Because training still uses paired residuals, we do not describe the absence of a direct cross-family target as “unsupervised learning.”

All four encoders and decoders have the same architecture: a linear projection, b pre-norm residual blocks, and a linear projection to the output. The reported configuration has latent width 4096, $b=1$, and internal width 4096. The second matrix in each residual block is zero-initialized, so the block is initially the identity and E and D begin learning from linear maps. The non-affine LN between them means that the complete cross-family map is not linear even at initialization. Zero initialization describes only the starting point and does not imply that the trained map is linear. The two parents contain 14.25B parameters in total, while the adapter contains 268.5M, or 1.9% of the parent total. We use “lightweight” in this relative sense and to refer to the shallow mapping depth, not to claim that the absolute parameter count is small.

The noise ξ is introduced to prevent the latent from satisfying the alignment term with a token-independent constant known as the *common mode*. The design rationale is that a constant cannot evade the noise. We fix $\sigma=0.4$ and do not perform an ablation of its effect.

3.4 Four paths and chimera construction

The four paths are RWKV self-recon, RWKV→Pythia, Pythia self-recon, and Pythia→RWKV. RWKV→Pythia and Pythia→RWKV are cross-family chimeras; RWKV self-recon and Pythia self-recon are same-family reconstruction controls that pass through the same boundary and adapter. The switch layer L uses zero-based indexing and denotes

*⁴ Allen Institute for AI, *open-instruct-pythia-6.9b-tulu* model card, <https://huggingface.co/allenai/open-instruct-pythia-6.9b-tulu>.

*⁵ The released configuration is available at <https://huggingface.co/allenai/open-instruct-pythia-6.9b-tulu/blob/main/config.json>; its `max_position_embeddings` value is 2048.

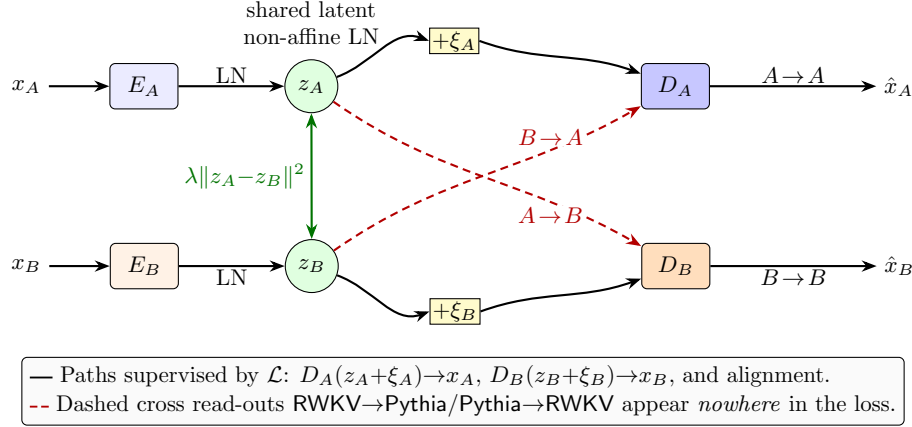


Fig. 1 Shared-latent adapter. Solid paths are self-reconstructions trained by Eq. (1); dashed paths are cross-family read-outs absent from the loss. Inputs are layerwise-standardized residuals, and noise is added only before decoding.

the last block executed by the prefix parent. Because block 0 is included, the prefix executes $L+1$ blocks. Here $h_A^L[t]$ is the unstandardized residual produced by block L of A at token position t . The symbol $\tilde{h}_B^L[t]$ denotes the analogous residual produced by block L of B . A tilde marks a residual translated into the other model’s coordinate system. Using $x_A^L[t] = \text{std}_A^L(h_A^L[t])$ and $x_B^L[t] = \text{std}_B^L(h_B^L[t])$, the two boundary maps are

$$\begin{aligned} \tilde{h}_B^L[t] &= \text{unstd}_B^L\left(D_B\left(\text{LN}\left(E_A\left(x_A^L[t]\right)\right)\right)\right), \\ \tilde{h}_A^L[t] &= \text{unstd}_A^L\left(D_A\left(\text{LN}\left(E_B\left(x_B^L[t]\right)\right)\right)\right). \end{aligned} \quad (2)$$

The translated residual $\tilde{h}_B^L[t]$ enters block $L+1$ of B , while $\tilde{h}_A^L[t]$ enters block $L+1$ of A . No training-time noise is added in Eq. (2). Execution has three stages: run blocks 0 through L of the prefix parent, translate the residual once, then run blocks $L+1$ through 31 and the language-model head of the suffix parent. Every chimera therefore executes 32 blocks in total, divided into $L+1$ prefix blocks and $31-L$ suffix blocks.

3.5 Evaluation metrics

We measure residual recovery with the coefficient of determination $R^2 = 1 - \text{SSE}/\text{SST}$. Here SSE is the sum of squared differences between predicted and target residuals, and SST is the sum of squared target residuals after subtracting the per-dimension token mean. A value of 1 means perfect recovery, 0 matches a predictor that always returns the per-dimension mean, and a negative value is worse than that mean predictor. We evaluate both cross-family maps $A \rightarrow B$ and $B \rightarrow A$, as well as the same-family round trips $A \rightarrow A$ and $B \rightarrow B$. The latter two control for information lost by encoding and decoding before the additional effect of crossing model families.

We compute R^2 separately at each layer by pooling the held-out tokens and all 4096 dimensions. Because SST is the total variation after removing the token mean of each dimension, dimensions with greater variance contribute more. Latent agreement is measured with centered correlation ρ_{ctr} . This is the Pearson correlation obtained after removing each

dimension’s token mean from z_A and z_B and flattening the resulting token-by-dimension arrays. Values near 1 mean that the two latents vary in the same direction, while 0 means no linear correlation. We also measure the fraction of latent energy that varies across tokens as $f = \|z - \bar{z}\|^2 / \|z\|^2$. Here $\bar{z}$ is the per-dimension token mean within a layer. We compute f separately for A and B and average the two values. Every metric is computed per layer, and we report the simple mean over all 32 layers.

We use two held-out sets. The training-time evaluation that supplies the headline values contains 1,000 windows, or 112,000 tokens. The per-layer analysis and shuffle control use 400 windows, or 44,800 tokens. Every window has 112 tokens. Because the evaluation sets differ, the 32-layer averages in the per-layer analysis, 0.561/0.592, do not exactly match the headline values, 0.559/0.590. The fitted R^2 over all layer correspondences is an in-sample ridge linear probe that is fitted and evaluated on the same 22,400 rows for every layer pair.

We evaluate the language function of the chimeras in three ways. First, we provide English prompts verbatim and generate samples by deterministic argmax at temperature 0. Generation stops after at most 40 new tokens or earlier only when the model emits EOS. Second, we score every answer option in ARC-Easy and SciQ by log likelihood and measure the accuracy of selecting the highest-scoring option. Third, we measure next-token cross-entropy on Alpaca and Wiki-Text to compare language modeling within and outside the training domain.

3.6 Reproduction Information

- Models: Raven 7B distributed by BlinkDL was converted to Hugging Face format and re-saved in fp16. The second model is the fp16 release `allenai/open-instruct-pythia-6.9b-tulu`. Both have 32 layers, $d=4096$, and the GPT-NeoX-20B tokenizer. The upstream revisions used during conversion or re-saving were not recorded for either parent. The release artifact

`parent_fingerprints.json` records SHA-256 fingerprints of the local parent weights used for adapter training.

- Adapter: $d_z=d=4096$, one residual block with internal width 4096, non-affine LN, latent-alignment weight $\lambda=1$, noise standard deviation $\sigma=0.4$, and 268.5M parameters. The second matrix of the residual block is zero-initialized. We use AdamW on $4\times V100$ GPUs with an effective batch size of 4096. The learning rate decreases from 10^{-3} to 10^{-5} and is multiplied by 0.5 whenever held-out $A \rightarrow B$ stops improving. The run uses random seed 0 for 416,000 iterations. We report iteration 415,000, which is best on held-out $A \rightarrow B$.
- Data: residuals come from the leading portion of the Alpaca training split in windows of 112 tokens, giving approximately 232k independent token positions. Evaluation windows begin at character position 2,000,000, a fixed offset independent of the training token count, so the two portions do not overlap. Standardization statistics are computed from the training residuals of each layer and reused for inverse standardization.
- Evaluation: held-out R^2 pools all tokens and dimensions within each layer, and we report the simple mean over the 32 layers. Multiple-choice evaluation scores the sequence obtained by appending each option to the prompt. We compute both the total log likelihood and its mean over post-prompt tokens. The full evaluation sets contain 2376 ARC-Easy test questions and 1000 SciQ test questions. SciQ is evaluated in closed-book form without the support passage. Differences between configurations are assessed with McNemar tests and paired bootstrap tests. For the RWKV suffix of `Pythia→RWKV`, options are always evaluated at batch 1. `Pythia→RWKV` is likewise fixed to the same distributed checkpoint as `RWKV→Pythia`, step 415,000 selected on held-out $A \rightarrow B$; no re-selection of checkpoints that saw $B \rightarrow A$ or `Pythia→RWKV` QA is performed. However, the two direction-fixed released runtimes take as default the $L=4$ whose reported test mean was highest over the four boundaries of each path after evaluation, with no independent deployment-selection split.
- Code and structured data: the repository for this work is <https://github.com/Katagiri-Hoshino-Lab/NinaXander>. It contains the experiment code, the raw data of each evaluation together with aggregate tables of the values reported in the paper, the reproduction procedure, and checks that automatically cross-verify the numbers in the text against the tabulated data.
- Hugging Face distributions: the inference artifacts as of July 30, 2026 were stored in the following three model repositories.
 - Shared adapter: <https://huggingface.co/Katagiri-Hoshino-Lab/ninaxander-raven7b-tulu69-adapter>
 - Pythia→RWKV (default $L=4$): <https://huggingface.co/Katagiri-Hoshino-Lab/ninaxander-tulu69-to-raven7b-best>
 - RWKV→Pythia (default $L=4$): <https://huggingface.co/Katagiri-Hoshino-Lab/ninaxander-raven7b-to-tulu69-best>

Katagiri-Hoshino-Lab/

`ninaxander-raven7b-tulu69-adapter`

- Pythia→RWKV (default $L=4$):

<https://huggingface.co/Katagiri-Hoshino-Lab/ninaxander-tulu69-to-raven7b-best>

- RWKV→Pythia (default $L=4$):

<https://huggingface.co/Katagiri-Hoshino-Lab/ninaxander-raven7b-to-tulu69-best>

Each repository includes model cards and notices in English, Japanese, and Simplified Chinese, SafeTensors, and a SHA-256 manifest. All distribution licenses follow the more restrictive parent, Tulu-Pythia, and use the AI2 AI Model License, which limits use to non-commercial research. Each NOTICE records attribution to the parent models.

4. Results

We first test whether corresponding layers can be aligned and how accurately each model’s residual can be recovered from the resulting representation. We then isolate the adapter’s nonlinearity and evaluate the generation, accuracy, and inference memory of the composed chimeras.

4.1 A Learnable Shared Representation Exists Between Corresponding Layers

Aligning two 7B instruction-following models that share a tokenizer, depth, and hidden width on Alpaca [29] instruction-formatted text lets an adapter with one residual block reach a centered latent correlation of $\rho_{ctr} = 0.901$ on unseen text. In this paper we claim a shared representation on the grounds of the centered correlation of 0.901 together with the fact that each decoder can reconstruct residuals of its own family at $A \rightarrow A$ 0.70 and $B \rightarrow B$ 0.71. Moreover, even the best *linear* maps fitted directly to the two cross-family tasks reach $A \rightarrow B$ 0.487 and $B \rightarrow A$ 0.535. These substantial linear baselines show that the expressive power of a deep nonlinear map alone cannot explain most of the observed predictability.

However, same-index layer pairs are not intrinsically special. Figure 2 compares raw residuals without an adapter across all 32×32 layer pairs. Linear CKA is weak throughout, with a diagonal mean of 0.05 and an off-diagonal mean of 0.03. Layer j of A is most similar to layer j of B in only 2 of 32 rows. Layer 0 is the most similar target in 14 rows, and the final layer is the most similar target in 12 rows. Same-index layers are also unexceptional in direct linear regressability. Per-layer linear maps fitted to all layer pairs attain fitted R^2 values from 0.62 to 0.94. Their diagonal mean is 0.87, compared with an off-diagonal mean of 0.84. The same-numbered layer is best in only 3 of 32 rows. Thus $\rho_{ctr} = 0.901$ is an alignment *learned and recovered on the selected same-index layer pairs*, not a correspondence intrinsic to matching layer numbers. We position this work as a case study that selects same-index layers from a broader regressability that exists across arbitrary layer pairs.

The alignment is *learnable*, but equal layer numbers do

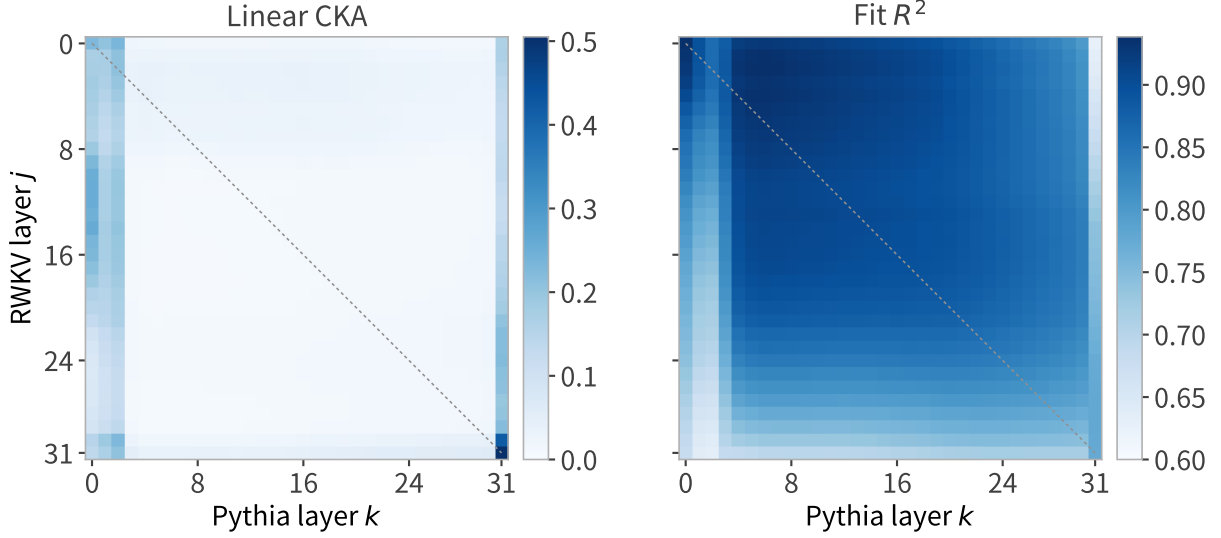


Fig. 2 All 32×32 layer correspondences of raw residuals. The left panel shows linear CKA and the right shows the fitted R^2 of per-layer linear maps. The dashed line marks equal layer indices; the colour scales differ.

not constitute a natural correspondence.

All numbers below come from a single training run of the same configuration. The latent width is $d_z=d=4096$, all 32 blocks are paired, the adapter has 268.5M parameters, and $\sigma=0.4$. A plateau schedule reduces the learning rate from 10^{-3} to a floor of 10^{-5} . The training signal consists of residuals from 2,075 windows of the Alpaca training split. Each window contains 112 tokens, giving approximately 232k independent token positions. Across all 32 layers, these residuals form 7.44M pairs, and training iterates over this pair set about 228 times. Reported values come from iteration 415,000, the deepest checkpoint at which held-out $A \rightarrow B$ is best. The Pythia $\rightarrow$ RWKV cross-family map attains $B \rightarrow A$ 0.590, slightly above $A \rightarrow B$ 0.559. Residuals on the RNN side are therefore slightly easier to recover from the Transformer latent. Both cross-family maps fall below the same-family reconstructions $A \rightarrow A$ 0.695 and $B \rightarrow B$ 0.711. Even with high latent alignment, error is already present in same-family reconstruction, and cross-family read-out adds further loss. We do not isolate whether this error originates in the encoder, latent normalization, decoder, or training-time noise.

A negative control confirms that the alignment and read-outs depend on a true token-by-token correspondence rather than distributional overlap. We keep the row order of A fixed and re-pair the residual rows of B with one random permutation. This collapses ρ_{ctr} from 0.901 $\rightarrow$ 0.000, $A \rightarrow B$ from 0.561 $\rightarrow$ -0.592, and $B \rightarrow A$ from 0.592 $\rightarrow$ -0.609. The same-family maps $A \rightarrow A$ and $B \rightarrow B$ are unchanged by construction. Neither cross-family read-out is therefore obtainable without the correct token correspondence.

In the following, z denotes either z_A or z_B , and $\bar{z}$ denotes its per-dimension token mean within the layer. The quantity ρ_{raw} is the Pearson correlation of z_A and z_B before subtracting these token means. The quantity align is the

mean squared distance between z_A and z_B represented by the third error term of Eq. (1), before multiplication by λ . Let f denote the fraction of the latent that varies across tokens. Under layer normalization without elementwise gains, the raw and centered correlations decompose as

$$\begin{aligned} f &= \frac{\|z - \bar{z}\|^2}{\|z\|^2}, \\ \rho_{\text{raw}} &= (1 - f) + f \rho_{\text{ctr}}, \\ \text{align} &= 2f(1 - \rho_{\text{ctr}}). \end{aligned} \quad (3)$$

The reported f is the mean of the varying fractions computed separately for A and B . Equation (3) holds exactly when the two fractions are equal and their common modes are parallel. At evaluation points after training has progressed, the measured relation holds to a precision of about 3×10^{-4} . It does not hold early in training; at iteration 0, the residual reaches 6.6×10^{-2} . Equation (3) shows that raw correlation contains both centered correlation and the contribution of a constant term. The fraction $1 - f$ comes from a constant that carries no information. Since f at the deepest point is 0.350, 65% of the latent’s energy is a token-independent constant, and this constant dominates the raw correlation. The centered ρ_{ctr} is 0.901, while the raw correlation is 0.965. Equation (3) reproduces this value exactly as $(1 - 0.350) + 0.350 \times 0.901 = 0.965$.

What f does and does not show

f is a second moment, not a measure of information, and the inferences it licenses run in one direction only. $f \rightarrow 0$ proves that the latent has collapsed to a constant, and hence that it carries no per-token information at all. A large f means only that the per-token latents are spread out in angular direction; under layer normalization without elementwise gains, f is strictly invariant to scaling. Spread arises equally from meaningful variation and from meaningless variation. It is therefore the read-out, not f , that can

establish that the varying component carries information. The read-out is a centered R^2 , so no token-independent predictor can exceed 0. The best constant scores exactly 0. Hence $B \rightarrow B = 0.711$ cannot be obtained from any constant, which is consistent with 65% of the latent’s energy being constant.

4.2 Both Cross Read-Outs Fall Below Same-Family Reconstruction in Every Layer

Fig. 3 shows the five measured diagnostic values.

$$\begin{aligned} \rho_{\text{ctr}} &= 0.901, \\ R^2(B \rightarrow B) &= 0.711, \quad R^2(A \rightarrow B) = 0.559, \\ R^2(A \rightarrow A) &= 0.695, \quad R^2(B \rightarrow A) = 0.590. \end{aligned}$$

ρ_{ctr} is a correlation coefficient, whereas the latter four are coefficients of determination; the two cannot be compared as quantities on the same scale. The conclusions of this section rest on two observations. First, among the R^2 values, $A \rightarrow B \leq B \rightarrow B$ and $B \rightarrow A \leq A \rightarrow A$ hold in all 32 layers. Second, the loss from the encoding and decoding round trip also appears in downstream performance. It is significant for Pythia self-recon at every switch layer on both tasks, and for RWKV self-recon at $L=16, 24$ on both tasks.

High latent correlation does not guarantee high reconstruction accuracy. RWKV→Pythia can be compared with Pythia self-recon, which passes through the same decoder D_B , and Pythia→RWKV with RWKV self-recon, which passes through the same decoder D_A ; the only difference is whether the input is a same-family or a cross-family latent. Both inequalities are empirical observations rather than theorems, but in either direction a substantial error is already present at the self-reconstruction stage, and improving only the cross-family step leaves that error in place.

At $d_z=d=4096$ the latent is as wide as the residual. The non-affine LN discards only two degrees of freedom per token, namely its mean and norm, so width or rank is unlikely to be a strong constraint on reconstruction. Even so, $B \rightarrow B$ remains at 0.711. Whether this is a ceiling set by the noise σ is unknown, and what is rate-limiting remains an open problem.

4.3 The Adapter Is Nonlinear on Both Cross Paths

Each of the adapter’s E and D is a single zero-initialized residual block sandwiched between two linear maps, so each begins training as a linear map. But that is a property before training, not after 415,000 iterations. We measured how linear the adapter at the deepest point actually is, for both RWKV→Pythia and Pythia→RWKV, on the same 5 representative layers and the same fit and held-out rows.

The best affine map mimicking the adapter’s own output explains 86.6% of the output variance for RWKV→Pythia and 88.0% for Pythia→RWKV. Hence the remaining 13.4%/12.0% of the output variance cannot be captured by the fitted affine maps. The trained map is clearly nonlinear on both paths.

The comparison is not an approximation of the adapter: it is a control obtained by *directly* fitting, per layer and separately for each path, the best affine map from the standardized input residuals to the standardized output residuals via ridge least squares. Evaluated on the held-out rows of the 5 representative layers $\{4, 10, 16, 22, 28\}$, the best direct affine map reaches 0.487 for RWKV→Pythia and 0.535 for Pythia→RWKV. The corresponding adapter, on the same rows, achieves 0.558 for RWKV→Pythia and 0.576 for Pythia→RWKV, so the nonlinear map’s increment is $+0.071/+0.041$. Because even a low-capacity affine map achieves substantial cross-family prediction on both paths, *the shared-representation claim is underwritten by this linear reachability*.

The separately fitted affine control. At each switch layer this control is fitted by ridge regression with an intercept and a regularization coefficient of 10^{-3} . The fit uses 40,096 rows of standardized residuals drawn from the held-out region of Alpaca that the adapter was not trained on. In Table 1, replacing the RWKV→Pythia adapter with the direct affine map collapses accuracy by 8–23 points. For Pythia→RWKV, by contrast, the gap is small, and on SciQ @8 the affine control is 3.4 points higher. This matters as a path difference, but because the two maps differ in training objective, data, and regularization, we do not treat it as a causal effect of nonlinearity.

A confound-controlled intervention: disabling the MLP branch of the same adapter. We therefore scale only the residual block of the same trained adapter by a coefficient α . This block supplies its sole learned nonlinearity, $\text{fc}_2(\text{GELU}(\text{fc}_1(\cdot)))$. Setting $\alpha=1$ leaves the map intact, while $\alpha=0$ gives *the identical map with the branch removed*. Sweeping representation R^2 over $\alpha = 0, .25, .5, .75, 1$ yields $-0.41, -0.75, 0.35, 0.52, 0.56$ for RWKV→Pythia and $-0.30, -0.66, 0.36, 0.53, 0.58$ for Pythia→RWKV. On both paths, removing the branch collapses performance all the way to negative R^2 , and moreover the sweep is non-monotonic. The two linear layers were trained in coordination with the residual block, and in this trained adapter, the nonlinear branch cannot be removed from the read-out of Pythia→RWKV any more than from RWKV→Pythia. This is a property of the trained map, and note that this paper does not compare against a linear adapter trained with the same objective. For multiple choice as well, we evaluate the same $\alpha = 0/1$ pair on identical questions in paired fashion.

The branch effect in Table 2 is $+11.1\text{--}+39.6$ points for RWKV→Pythia and $+11.2\text{--}+36.6$ points for Pythia→RWKV, and an exact McNemar test on identical questions gives $p < 2 \times 10^{-9}$ in all 16 comparisons. Hence it is directly confirmed that removing the nonlinear branch from this trained adapter collapses representation and downstream accuracy alike on both RWKV→Pythia and Pythia→RWKV. This, however, is a property of removing the branch alone from a jointly trained map; it does not show that nonlinearity is generally necessary for connecting the families. At least for Pythia→RWKV accuracy, the separately fitted

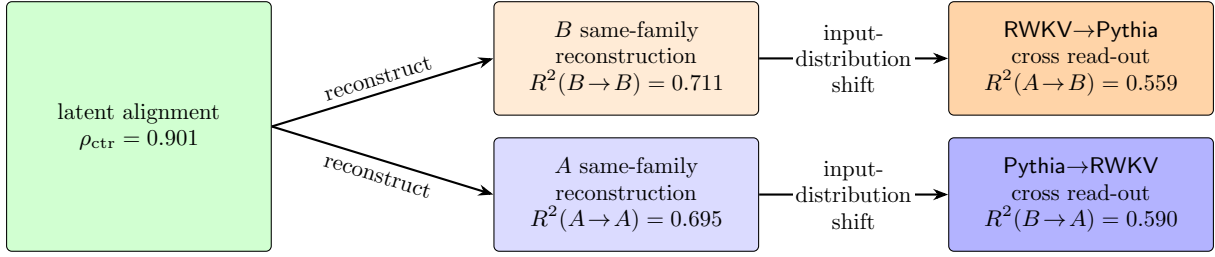


Fig. 3 The four read-outs from the shared latent. Paths of the same colour share a decoder.

Table 1 Length-normalized accuracy of the trained adapters and affine controls fitted directly to each path. Values are percentages and use the same questions, switch layer, and suffix.

L	ARC-Easy				SciQ			
	RWKV→Pythia		Pythia→RWKV		RWKV→Pythia		Pythia→RWKV	
	adapter	affine	adapter	affine	adapter	affine	adapter	affine
4	59.6	41.9	62.2	60.2	70.9	47.5	69.2	69.6
8	59.0	43.1	57.3	56.0	67.7	49.1	64.6	68.0
16	48.6	34.8	57.5	57.5	54.5	35.9	68.0	69.6
24	47.0	39.4	59.6	59.2	51.1	42.7	69.9	68.2

Table 2 Length-normalized accuracy after disabling the nonlinear branch of the same adapter. Values are percentages; each cell shows the change $\alpha : 1 \rightarrow 0$ from the full map to the branch-disabled map.

L	ARC-Easy		SciQ	
	RWKV→Pythia	Pythia→RWKV	RWKV→Pythia	Pythia→RWKV
4	59.6 → 31.9	62.2 → 31.8	70.9 → 31.3	69.2 → 32.6
8	59.0 → 29.9	57.3 → 35.5	67.7 → 31.4	64.6 → 37.0
16	48.6 → 28.5	57.5 → 43.7	54.5 → 31.7	68.0 → 47.3
24	47.0 → 34.8	59.6 → 48.4	51.1 → 40.0	69.9 → 49.1

affine control differs little from the adapter. The difference ranges from -3.4 to $+2.0$ points across the 8 cells, and Table 1 shows that the adapter does not consistently outperform it.

4.4 Syntactically Plausible Generation, Degraded Factuality

Here we actually compose both cross-family paths at 7B. At switch layer L , we run the prefix parent from block 0 through L for a total of $L+1$ blocks. We translate its residual *once* into the other space with Eq. (2), then run the suffix parent’s blocks from $L+1$ onward together with its language-model head. This is not a per-block round trip. Thus RWKV→Pythia ($L=16$) uses 17 RWKV blocks and 15 Pythia blocks, and Pythia→RWKV ($L=16$) the reverse; neither is an exact half.

We insert the same deepest-point adapter at $L=4$ for both RWKV→Pythia and Pythia→RWKV, and greedily decode the same four English prompts at temperature 0 with at most 40 new tokens. $L=4$ is the boundary at which the two-task average is largest among the four boundaries in each cross-family direction, and it is the common boundary at which both directions enter the non-dominated set on the SciQ KV-accuracy plane. Figure 5 shows this relation. This is, however, a representative-point selection made after see-

ing the same reported tasks, not an independent selection criterion for generation. This comparison is not an exhaustive generation benchmark, but we observe the outputs of both parent models and both chimera models under identical conditions.

From these case studies we observe that both RWKV→Pythia and Pythia→RWKV can generate syntactically plausible output. Both answer the capital of France correctly, and both start the primary colors correctly. RWKV→Pythia also answers the black-hole prompt correctly in one sentence, so instruction-following sentence generation itself does succeed. On the other hand, fluency can be preserved while content or instruction format is lost. Both directions complete the water prompt incorrectly with “hydrogen”, and Pythia→RWKV drifts into a dialogue format after correctly answering the primary colors and explains the black hole circularly. Thus even cross read-outs of 0.559/0.590 are not sufficient for factuality. A more faithful chimera requires improving both cross read-outs and same-family reconstruction.

4.5 Function and Inference-Memory Comparison Against the Parents

Assessing the practical value of NinaXander requires a functional comparison against the frozen parents, not

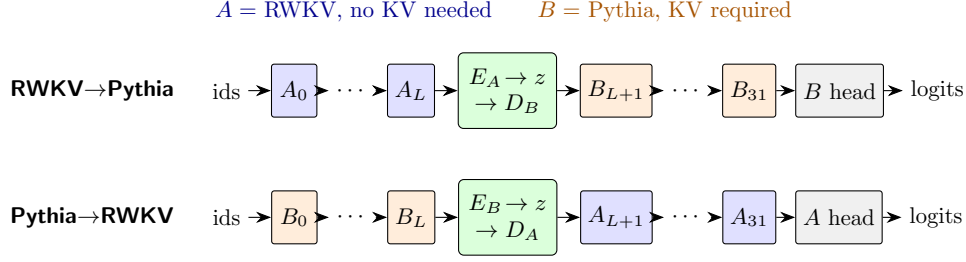


Fig. 4 Construction of the RWKV→Pythia/Pythia→RWKV chimeras. The prefix runs through block L , its residual is translated once, and the suffix starts at block $L+1$.

Table 3 Generations from both parents and the $L=4$ RWKV→Pythia/Pythia→RWKV chimeras on identical prompts. Decoding uses temperature 0, at most 40 new tokens, and early stopping only on EOS.

Prompt (verbatim)	pure Pythia	RWKV→Pythia cross read-out $R^2=0.559$	pure RWKV	Pythia→RWKV cross read-out $R^2=0.590$
### Instruction: What is the capital of France?	"The capital of France is Paris."	" The capital of France is Paris. " ✓	"Paris."	" Paris. " ✓
### Response: The three primary colors are	"red, yellow, and blue. These colors are often used in combination to create a wide variety of colors."	" red, yellow, and blue. " ✓	Correct with "red, blue, and yellow.", then drifts into a Q&A asking for the opposite color of red (green)	Correct with "red, blue, and yellow.", then drifts into a Bob/Alice dialogue about green
Question: Water is made of hydrogen and Answer:	"Hydrogen is made of hydrogen and oxygen."	"Hydrogen." incorrect	"hydrogen." incorrect	"hydrogen." incorrect
### Instruction: Explain what a black hole is in one sentence. ### Response:	"A black hole is a region of spacetime where gravity is so strong that nothing, not even light, can escape its pull."	"A black hole is a region in space where gravity is so strong that nothing, not even light, can escape its pull." ✓	"A black hole is a region of space where the gravitational pull is so strong that nothing, not even light, can escape."	"A black hole is a type of celestial object that is formed when a star collapses and collapses into a black hole." <i>circular</i>

internal-representation metrics alone. We measure three aspects: how much performance is retained relative to the stronger parent, whether the weaker parent is surpassed, and how much KV cache is saved in exchange.

This comparison is run in a multiple-choice format that computes the log-likelihood of each option and selects the maximum. Because no generation, string matching, or subjective judgment is involved, the result is uniquely determined. We use the full evaluation sets: the ARC-Easy test set contains 2376 questions [30], and the SciQ test set contains 1000 questions [31]. We evaluate the $d_z=4096$ adapter at its deepest reported point, iteration 415,000, at four switch positions. Between-configuration comparisons on the same question set are paired data, so differences are assessed with paired tests. We use McNemar’s exact test and paired bootstrap rather than independent per-cell binomial standard errors. Because the options differ in length and sum vs. length-normalized scoring can disagree in ranking, both are stored in the structured data, and the main text uses length-normalized scoring for reporting and testing. Moreover, SciQ is evaluated in a closed-book format that supplies only the question and answer options, not the accompanying support passages. The SciQ values in this paper are not directly comparable to published values under the standard

protocol that provides support.

Six observations follow from the joint comparison of the four paths. First, even with the prefix and suffix from different families, both RWKV→Pythia and Pythia→RWKV can actually answer the questions. All cross configurations far exceed chance, reaching RWKV→Pythia 70.9% and Pythia→RWKV 69.9% on SciQ. Second, the depth effect is path-dependent. RWKV→Pythia decreases monotonically over $L = 4 \rightarrow 24$, SciQ 70.9 $\rightarrow$ 67.7 $\rightarrow$ 54.5 $\rightarrow$ 51.1 and ARC-Easy 59.6 $\rightarrow$ 59.0 $\rightarrow$ 48.6 $\rightarrow$ 47.0, whereas Pythia→RWKV is non-monotonic, SciQ 69.2 $\rightarrow$ 64.6 $\rightarrow$ 68.0 $\rightarrow$ 69.9 and ARC-Easy 62.2 $\rightarrow$ 57.3 $\rightarrow$ 57.5 $\rightarrow$ 59.6. Local read-out at the same four layers is RWKV→Pythia 0.570/0.555/0.575/0.567 and Pythia→RWKV 0.552/0.517/0.604/0.608, so the switch layer or R^2 alone does not determine the ranking. Third, the stronger parent is not surpassed on either path. All 8 cross configurations fall below Tulu-Pythia on both tasks, and the differences are significant under McNemar tests on the same questions in every case. Fourth, the difference against RWKV is likewise path- and task-dependent. RWKV→Pythia ($L=4$) exceeds RWKV by +3.9 points on SciQ with $p=0.018$, but is −2.2 points on ARC-Easy with $p=0.042$. Under sum scoring, this advantage disappears. The sum-score difference

Table 4 Length-normalized accuracy of the four paths in percent. The largest value in each row for each task is bold.

L	RWKV self-recon		RWKV→Pythia		Pythia self-recon		Pythia→RWKV	
	ARC	SciQ	ARC	SciQ	ARC	SciQ	ARC	SciQ
4	62.8	68.5	59.6	70.9	62.2	74.8	62.2	69.2
8	59.8	66.7	59.0	67.7	62.7	74.8	57.3	64.6
16	54.8	59.8	48.6	54.5	57.4	71.1	57.5	68.0
24	52.2	57.9	47.0	51.1	59.4	70.1	59.6	69.9

for RWKV→Pythia ($L=4$) is -2.7 points on SciQ, while the differences for Pythia→RWKV ($L=4$) are -4.6 points on ARC and -5.7 points on SciQ. The verdict against the weaker parent therefore depends on the choice of length normalization. Pythia→RWKV does not significantly exceed RWKV at any boundary. Pythia→RWKV ($L=4$) is statistically equivalent at ARC with a difference of $+0.4$ points and $p=0.66$, and at SciQ with a difference of $+2.2$ points and $p=0.14$. Fifth, the difference against the separately fitted affine controls is also path-dependent. On RWKV→Pythia the adapter is 8–23 points higher in every cell, whereas on Pythia→RWKV the differences are small, and at SciQ @8 the affine is 3.4 points higher. That is, the trained adapter does not always outperform an affine mapping. Sixth, the gap to the same-family controls is not translation error alone. With the B suffix, accuracy decreases in the order pure Pythia, Pythia self-recon, RWKV→Pythia, with Pythia self-recon–RWKV→Pythia gaps of SciQ 4–19 points and ARC 3–12 points. With the A suffix, by contrast, Pythia→RWKV conversely exceeds RWKV self-recon at deep boundaries, and the differences at $L = 16/24$ are ARC $+2.7/+7.4$ points and SciQ $+8.2/+12.0$ points. The same-family controls isolate the loss from the encode–decode round trip, but because the gap to the cross paths contains both the prefix parent’s capability and translation mismatch, it cannot be read as a universal “translation cost.”

Depth dependence differs between RWKV→Pythia and Pythia→RWKV

As Table 4 shows, at $L=16, 24$ Pythia→RWKV exceeds the same-layer RWKV→Pythia by $+8.9/+12.6$ points on ARC and $+13.5/+18.8$ points on SciQ. McNemar tests on the same questions give $p < 3 \times 10^{-15}$ in every case. At $L=4$, on the other hand, the difference is $+2.6$ points on ARC with $p=0.015$, but -1.7 points on SciQ with $p=0.28$. The path difference is therefore not uniform at shallow boundaries. Every Pythia→RWKV configuration falls significantly below Pythia, and no Pythia→RWKV configuration significantly exceeds RWKV; Pythia→RWKV ($L=4$) is equivalent at ARC 62.2 vs. 61.8 with $p=0.66$ and SciQ 69.2 vs. 67.0 with $p=0.14$.

The best affine mapping for Pythia→RWKV is a control fitted separately from the adapter at each switch layer. At ARC @4 the adapter is $+2.0$ points higher with $p=0.005$, but most of the other cells show no difference. At SciQ @8 the affine is conversely $+3.4$ points higher with $p=0.003$. Separately from this between-mapping comparison, the intervention that sets the nonlinear branch of the same adapter

to $\alpha = 0$ is executed for RWKV→Pythia/Pythia→RWKV at the same four layers and the same questions, and the causal contribution of the nonlinear branch is assessed from those paired results. The comparison with the RWKV self-recon control likewise replaces the RWKV prefix simultaneously with a Pythia prefix and cross translation, and thus does not isolate translation error alone.

The memory effect of composition: the KV cache disappears together with the Transformer layers

Performance and inference memory must be evaluated jointly, because the two families do not have the same inference-time cost. Transformer layers cache key/value per token, whereas an RNN layer carries only a recurrent state that is $O(1)$ in sequence length. RWKV→Pythia retains only $31-L$ Transformer layers, and Pythia→RWKV only $L+1$. What is reduced is the KV cache, not the resident weights: the adapter itself occupies 0.50 GiB in fp16, or about 0.25 GiB if only one direction is kept resident, which is comparable to the reduced KV.

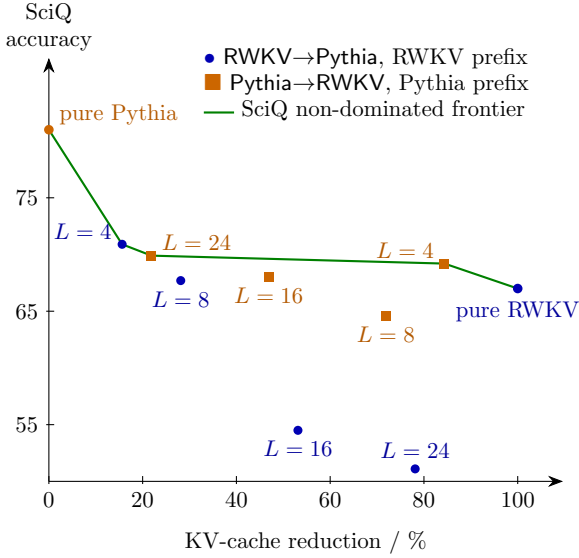
Including RWKV→Pythia/Pythia→RWKV, Pythia→RWKV ($L=4$) reduces the Transformer layers from $27 \rightarrow 5$ relative to RWKV→Pythia ($L=4$) while nearly preserving the two-task average, $65.25\% \rightarrow 65.70\%$. The recurrent state of the replacing RWKV is 64 KiB per layer. This value is computed from the reference implementation, which holds 2 of its 5 state vectors in fp16 and 3 in fp32. The recurrent-state size is independent of context length by construction. However, pure RWKV has zero KV and the ranking is task-dependent, so Fig. 5 is presented as a memory–accuracy frontier restricted to SciQ.

Adding the Pythia-prefix path yields the high-reduction point Pythia→RWKV ($L=4$). This point does not appear with the RWKV-prefix path alone. Meanwhile, pure RWKV still has zero KV, and Pythia→RWKV ($L=4$) does not significantly exceed RWKV either. The benefit is therefore the ability to create *additional trade-off points* between the frozen parents, not that the new configurations uniformly dominate the parents.

The results are qualified: function is preserved, but this is not a uniform improvement in accuracy. Both RWKV→Pythia and Pythia→RWKV fall significantly below the strong Pythia. RWKV→Pythia exceeds RWKV only on SciQ, and Pythia→RWKV does not significantly exceed RWKV at any boundary. The main benefit obtained is that Pythia→RWKV ($L=4$) removes 84.4% of the Transformer KV while retaining part of Pythia’s accuracy.

Table 5 Accuracy and inference memory for both paths. KV is measured; the RWKV recurrent state is independent of context length.

Configuration	Transformer layers	KV / token	KV @ 4k context	Reduction	ARC / SciQ
pure Tulu-Pythia	32	512 KiB	2.00 GiB	0%	65.3 / 81.0
RWKV→Pythia ($L=4$)	27	432 KiB	1.69 GiB	15.6%	59.6 / 70.9
RWKV→Pythia ($L=8$)	23	368 KiB	1.44 GiB	28.1%	59.0 / 67.7
RWKV→Pythia ($L=16$)	15	240 KiB	0.94 GiB	53.1%	48.6 / 54.5
RWKV→Pythia ($L=24$)	7	112 KiB	0.44 GiB	78.1%	47.0 / 51.1
Pythia→RWKV ($L=24$)	25	400 KiB	1.56 GiB	21.9%	59.6 / 69.9
Pythia→RWKV ($L=16$)	17	272 KiB	1.06 GiB	46.9%	57.5 / 68.0
Pythia→RWKV ($L=8$)	9	144 KiB	0.56 GiB	71.9%	57.3 / 64.6
Pythia→RWKV ($L=4$)	5	80 KiB	0.31 GiB	84.4%	62.2 / 69.2
pure RWKV-Raven	0	0	0	100%	61.8 / 67.0

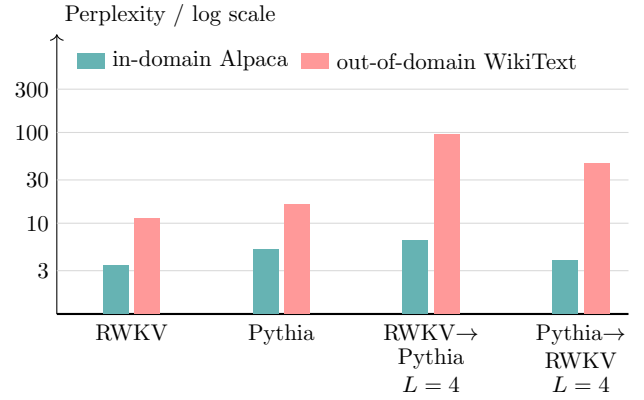

Fig. 5 SciQ accuracy and measured KV reduction for RWKV→Pythia/Pythia→RWKV. The green line joins the non-dominated points across both paths.

Translation is domain-specific

The accuracies above were measured on short questions close to instruction format. KV-cache savings matter for long contexts, and there we must measure how well the chimera preserves language modeling on a text corpus from a different domain. We use WikiText-103 [32] as the out-of-domain corpus. At context 512, RWKV→Pythia ($L=4$) has perplexity 6.40 on Alpaca and 93.87 on WikiText-103. Pythia→RWKV ($L=4$) improves in both domains at 3.85/45.00, but WikiText still remains far from the parents' 11.33/16.22. Extending the context to 2048, Pythia→RWKV ($L=4$) reaches Alpaca 3.50 and WikiText 34.78, improving rather than collapsing. The main cause is therefore not context length but *domain shift*. Pythia→RWKV mitigates the shift, but WikiText 34.78 is 4.2 times the better parent RWKV's 8.27 and its perplexity remains high.

5. Discussion

In our experiments, a shared representation with $\rho_{ctr} = 0.90$ can be learned between corresponding layers of frozen heterogeneous families. However, the scope of this claim is limited. We measure the correspondence under favorable conditions: an identical tokenizer, width, layer count, and


Fig. 6 Perplexity on Alpaca and WikiText-103 at context 512. The vertical axis is logarithmic.

input, together with a domain close to Alpaca. The shuffled control rules out token-independent distribution matching, but we do not disentangle whether the observed correspondence stems from semantics, vocabulary, position, or syntax. Section 4.5 shows that perplexity degrades on out-of-domain WikiText. This result leaves generalization beyond the training domain in question. We therefore do not claim a “general shared *semantic* space.” Our claim is limited to a shared space of token-aligned representations that is learnable under favorable conditions.

A functioning composition does not establish that the two sides represent the same information. Smith et al. [33] successfully stitch across different tasks and modalities, and even from representations derived from clustered random noise. Their stitching layers are trained on downstream task loss. Balogh and Jelasity [34] show that this regime rates non-corresponding and out-of-distribution representations highly. Our adapter instead learns by directly matching corresponding residuals, which the same study finds balances structural and functional requirements. Even so, Smith et al. obtain spurious alignment with a low-capacity stitch, while our adapter is considerably more expressive. Section 4.3 shows that a low-capacity affine map already achieves substantial cross-family prediction. This result is our main reason for concluding that adapter capacity alone does not manufacture the observed alignment.

High latent correlation also does not guarantee high read-out accuracy. The presence of a shared representation

and its decodability must be evaluated separately. Zhang and Xin independently report the same distinction in their preprint [35]. Their linear bridge between Pythia models reaches a normalized R^2 of 0.883 offline, yet degrades multi-hop reasoning when its mapped activation replaces the receiver’s activation. They conclude that the objective must include causal use by the receiver. Our adapter is likewise trained only on activation reconstruction and falls within the scope of this criticism. Equation (2) does include per-layer inverse standardization, which restores the receiver’s native scale, whereas their bridge trains under a normalized objective. We do not isolate whether this structural difference is what matters. A chimera built on this space functions, but at a loss, and in the generation samples both paths remain fluent while erring in content or form. A competitive chimera requires identifying the factors that bottleneck $B \rightarrow B$, training across domains, and selecting complementary parent models on the pretraining side.

6. Limitations

The evaluation tasks are exposed. Tulu-Pythia is instruction-tuned on a subset of FLAN V2, whose task inventory includes ARC and SciQ. Verbatim matching against the released components finds 19 ARC-Easy and 4 SciQ evaluation items. The matching procedure cannot detect paraphrases or formatting changes. It also excludes questions shorter than 6 words to avoid chance collisions; this exclusion covers 40 ARC-Easy and 11 SciQ questions. The detected counts are therefore lower bounds. Removing the matched items from the per-item correctness records changes no configuration’s accuracy by more than 0.22 points and changes none of the comparisons in this paper. The same items also become harder or easier in the same direction for RWKV, which had no such exposure. Its change is -9.2 points on ARC and $+8.0$ points on SciQ, so the gap reflects item difficulty rather than exposure. Nevertheless, ARC-Easy and SciQ remain task families already present in the suffix parent’s training. Our values should be read as capability retained through composition, not capability newly acquired.

Adapter training uses only a single random seed. Performance is measured on the full evaluation sets, which contain 2376 ARC-Easy questions and 1000 SciQ questions. Differences between configurations were assessed on the same questions with McNemar’s exact test and paired bootstrap. RWKV→Pythia exceeds the weaker parent only on SciQ and only under the length-normalized score. The advantage does not hold under the sum score. Pythia→RWKV does not significantly exceed RWKV at any boundary. We evaluate 2 tasks $\times$ 2 cross paths $\times$ 4 switches plus multiple controls, without correction for multiple comparisons. The identical-adapter intervention on the nonlinear branch was performed at the same layers and questions for both RWKV→Pythia/Pythia→RWKV, but it is an intervention on a single checkpoint and may not generalize to different adapter training conditions.

Moreover, the composition always has only a single boundary, and we did not measure how translation error accumulates across multiple boundaries. Furthermore, the chimera evaluation is likelihood-based, which measures the ability to *select* the correct option and does not directly measure the ability to *generate*.

The four generation samples for each of RWKV→Pythia and Pythia→RWKV are only a qualitative supplement, not a comprehensive evaluation of generation ability. In addition, the translation is *specialized to the training domain* and degrades substantially on WikiText. Long-context cross-entropy is measured at contexts 512, 1024, and 2048. Tulu-Pythia has a configured maximum of 2048 positions, and both cross-family paths include this model, so we use 2048 as their common evaluation limit. Measurements beyond the converted RWKV configuration’s value of 1024 are extrapolations beyond its configured length. We also cannot recover the upstream training context claimed by BlinkDL because the source filename and revision were not retained. These measurements concern language-modeling quality and do not directly measure reading comprehension that requires long-range dependencies. Ablations over σ , λ , latent width, the number of residual blocks, and the amount of training data, as well as alignment on a same-family pair with an identical architecture, were not conducted in this 32-layer configuration and are left for future work.

7. Conclusion

This work built NinaXander, a family of chimera models that connect frozen blocks from different pretraining families at a single boundary through one shared-latent adapter, and measured on RWKV-4-Raven-7B and Tulu-Pythia-6.9b what does and does not hold for cross-family composition, from the level of the representation down to downstream function. Under the favorable conditions of a shared tokenizer, width, depth, and training domain, the alignment between corresponding layers is *learnable*: the centered latent correlation reaches 0.901, while shuffling the row correspondence collapses it to 0.000. Since the CKA of the raw representations does not peak at same-numbered layers and performance degrades out of domain, what is shown is not a general semantic space but a token-aligned representation learnable under favorable conditions. Error is already present at the same-family reconstruction stage, before the cross-family step. The cross read-outs fall below, in every layer, the same-family controls that use the same respective decoders, and we do not isolate whether that error originates in the encoder, the latent normalization, the decoder, or the training-time noise. Removing the nonlinear branch from the trained adapter collapses both cross read-outs to negative R^2 , but that is a property of removing the branch alone from a jointly trained map and does not show that nonlinearity is generally necessary for connecting the families; for Pythia→RWKV, indeed, the separately fitted affine control differs little from the adapter. On the functional side, training the adapter once yields 8 configurations (4

switch positions $\times$ 2 directions) without per-configuration retraining, and the composed chimeras produce syntactically plausible generations and perform multiple choice. `Pythia` $\rightarrow$ `RWKV` ($L=4$) keeps only 5 Transformer layers, cutting the KV cache by 84.4% while retaining ARC-Easy 62.2% and SciQ 69.2%. What is reduced, however, is the KV cache and not the resident weights: the adapter itself occupies 0.50 GiB in fp16, or about 0.25 GiB if only one direction is kept resident, which adds a resident cost comparable to the reduced KV of 0.31 GiB at a 4k context. In the generation samples both paths remain fluent while erring in content or form, and no configuration surpasses the stronger parent Tulu-Pythia. `Pythia` $\rightarrow$ `RWKV` ($L=4$) does not significantly exceed `RWKV` either: it is statistically equivalent at ARC 62.2 vs. 61.8 ($p=0.66$) and SciQ 69.2 vs. 67.0 ($p=0.14$). The default $L=4$ is itself a post-hoc selection on the reported tests, with no independent deployment-selection split.

The open problems follow from the same measurements. The largest weakness is generalization out of domain: at context 2048, `Pythia` $\rightarrow$ `RWKV` ($L=4$) has a WikiText perplexity of 34.78, still far from the 8.27 of the better parent `RWKV`. Alongside it remain identifying what limits $B \rightarrow B$ at 0.711, training across domains, selecting complementary parent models on the pretraining side, measuring how translation error accumulates across multiple boundaries, ablations over σ , λ , the latent width, the number of residual blocks, and the amount of training data, and reproduction across several random seeds. What this work shows is not a competitive new model but that post-hoc recomposition of frozen assets is partly possible and that additional trade-off points can be created between the frozen parents; describing the conditions and limits of that route as measurements is the contribution of this paper.

References

- [1] Lenc, K. and Vedaldi, A.: Understanding Image Representations by Measuring Their Equivariance and Equivalence, *Int. J. Comput. Vision*, Vol. 127, No. 5, pp. 456–476 (online), DOI: 10.1007/s11263-018-1098-y (2019).
- [2] Bansal, Y., Nakkiran, P. and Barak, B.: Revisiting model stitching to compare neural representations, *Proceedings of the 35th International Conference on Neural Information Processing Systems*, NIPS '21, Red Hook, NY, USA, Curran Associates Inc. (2021).
- [3] Csizsárik, A., Körösi-Szabó, P., Matszangosz, A., Papp, G. and Varga, D.: Similarity and Matching of Neural Network Representations, *Advances in Neural Information Processing Systems* (Ranzato, M., Beygelzimer, A., Dauphin, Y., Liang, P. and Vaughan, J. W., eds.), Vol. 34, Curran Associates, Inc., pp. 5656–5668 (online), available from (https://proceedings.neurips.cc/paper_files/paper/2021/file/2cb274e6ce940f47beb8011d8ecb1462-Paper.pdf) (2021).
- [4] Hu, B., Li, S., Agarwal, S., Lee, M., Jajoo, A., Li, J., Xu, L., Kim, G.-W., Kim, D., Xu, H., Zhang, A. and Akella, A.: StitchLLM: Serving LLMs, One Block at a Time, *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 1: Long Papers)* (Che, W., Nabende, J., Shutova, E. and Pilehvar, M. T., eds.), Vienna, Austria, Association for Computational Linguistics, pp. 26887–26903 (online), DOI: 10.18653/v1/2025.acl-long.1305 (2025).
- [5] Zhang, Q., Bhargava, P., Bi, C., Cai, C. X., Foerster, J. N., Fu, J., Koura, P. S., Silva, R., Shen, S., Dinan, E., Gururangan, S. and Lewis, M.: BTS: Harmonizing Specialized Experts into a Generalist LLM, *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing* (Christodoulopoulos, C., Chakraborty, T., Rose, C. and Peng, V., eds.), Suzhou, China, Association for Computational Linguistics, pp. 6816–6834 (online), DOI: 10.18653/v1/2025.emnlp-main.347 (2025).
- [6] Bansal, R., Samanta, B., Dalmia, S., Gupta, N., Ganapathy, S., Bapna, A., Jain, P. and Talukdar, P.: LLM Augmented LLMs: Expanding Capabilities through Composition, *The Twelfth International Conference on Learning Representations*, (online), available from (<https://openreview.net/forum?id=jjA401vJRz>) (2024).
- [7] Roberts, N., Guo, S., Gao, Z., GNVV, S. S. S. N., Crompt, S., Wu, C., Duan, C. and Sala, F.: Pretrained Hybrids with MAD Skills, *Second Conference on Language Modeling*, (online), available from (<https://openreview.net/forum?id=8xSbwT3763>) (2025).
- [8] Moschella, L., Maiorca, V., Fumero, M., Norelli, A., Locatello, F. and Rodolà, E.: Relative representations enable zero-shot latent space communication, *The Eleventh International Conference on Learning Representations*, (online), available from (<https://openreview.net/forum?id=Src-nwieGJ>) (2023).
- [9] Maiorca, V., Moschella, L., Norelli, A., Fumero, M., Locatello, F. and Rodolà, E.: Latent space translation via semantic alignment, *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, Curran Associates Inc. (2023).
- [10] Grant, S.: Model Alignment Search, *Second Workshop on Representational Alignment at ICLR 2025*, (online), available from (<https://openreview.net/forum?id=8uNenpoG1h>) (2025).
- [11] Chen, A., Merullo, J., Stolfo, A. and Pavlick, E.: Transferring Linear Features Across Language Models With Model Stitching, *The Thirty-ninth Annual Conference on Neural Information Processing Systems*, (online), available from (<https://openreview.net/forum?id=Qvvy0X63Fv>) (2025).
- [12] Flamant, C., Ghai, U. and Shimizu, K.: The Bicausal Model: Bidirectional Hidden-State Coupling Between Parallel Language Models, arXiv:2605.11167, <https://arxiv.org/abs/2605.11167> (2026).
- [13] Goldstein, D., Obeid, F., Alcaide, E., Song, G. and Cheah, E.: GoldFinch: High Performance RWKV/Transformer Hybrid with Linear Pre-Fill and Extreme KV-Cache Compression, arXiv:2407.12077, <https://arxiv.org/abs/2407.12077> (2024).
- [14] Oozeer, N. F., Nathawani, D., Prakash, N., Lan, M., HARRASSE, A. and Abdullah, A.: Activation Space Interventions Can Be Transferred Between Large Language Models, *Forty-second International Conference on Machine Learning*, (online), available from (<https://openreview.net/forum?id=HX0icJsmMQ>) (2025).
- [15] Kornblith, S., Norouzi, M., Lee, H. and Hinton, G.: Similarity of Neural Network Representations Revisited, *Proceedings of the 36th International Conference on Machine Learning* (Chaudhuri, K. and Salakhutdinov, R., eds.), Proceedings of Machine Learning Research, Vol. 97, PMLR, pp. 3519–3529 (online), available from (<https://proceedings.mlr.press/v97/kornblith19a.html>) (2019).
- [16] Raghu, M., Gilmer, J., Yosinski, J. and Sohl-Dickstein, J.: SVCCA: singular vector canonical correlation analysis for deep learning dynamics and interpretability, *Proceedings of the 31st International Conference on Neural Information Processing Systems*, NIPS'17, Red Hook, NY, USA, Curran Associates Inc., pp. 6078–6087 (2017).
- [17] Morcos, A. S., Raghu, M. and Bengio, S.: Insights on representational similarity in neural networks with canonical correlation, *Proceedings of the 32nd International Conference on Neural Information Processing Systems*, NIPS'18, Red Hook, NY, USA, Curran Associates Inc., pp. 5732–5741 (2018).
- [18] Wang, J., Ge, X., Shu, W., Tang, Q., Zhou, Y., He, Z. and Qiu, X.: Towards Universality: Studying Mechanistic Similarity Across Language Model Architectures, *The Thirteenth International Conference on Learning Representations*, (online), available from (<https://openreview.net/forum?id=2J18i8T0oI>) (2025).
- [19] Wortsman, M., Ilharco, G., Gadre, S. Y., Roelofs, R., Gontijo-Lopes, R., Morcos, A. S., Namkoong, H., Farhadi, A., Carmon, Y., Kornblith, S. and Schmidt, L.: Model soups: averaging weights of multiple fine-tuned models improves

- accuracy without increasing inference time, *Proceedings of the 39th International Conference on Machine Learning* (Chaudhuri, K., Jegelka, S., Song, L., Szepesvari, C., Niu, G. and Sabato, S., eds.), Proceedings of Machine Learning Research, Vol. 162, PMLR, pp. 23965–23998 (online), available from (<https://proceedings.mlr.press/v162/wortsman22a.html>) (2022).
- [20] Yadav, P., Tam, D., Choshen, L., Raffel, C. and Bansal, M.: TIES-MERGING: resolving interference when merging models, *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, Curran Associates Inc. (2023).
- [21] Akiba, T., Shing, M., Tang, Y., Sun, Q. and Ha, D.: Evolutionary optimization of model merging recipes, *Nature Machine Intelligence*, Vol. 7, No. 2, pp. 195–204 (online), DOI: 10.1038/s42256-024-00975-8 (2025).
- [22] Wan, F., Huang, X., Cai, D., Quan, X., Bi, W. and Shi, S.: Knowledge Fusion of Large Language Models, *The Twelfth International Conference on Learning Representations*, (online), available from (<https://openreview.net/forum?id=jiDsk12qc2>) (2024).
- [23] Gao, L., Biderman, S., Black, S., Golding, L., Hoppe, T., Foster, C., Phang, J., He, H., Thite, A., Nabeshima, N., Presser, S. and Leahy, C.: The Pile: An 800GB Dataset of Diverse Text for Language Modeling, arXiv:2101.00027, <https://arxiv.org/abs/2101.00027> (2020).
- [24] Peng, B., Alcaide, E., Anthony, Q., Albalak, A., Arcadinho, S., Biderman, S., Cao, H., Cheng, X., Chung, M., Derczynski, L., Du, X., Grella, M., Gv, K., He, X., Hou, H., Kazienko, P., Kocon, J., Kong, J., Koptyra, B., Lau, H., Lin, J., Mantri, K. S. I., Mom, F., Saito, A., Song, G., Tang, X., Wind, J., Woźniak, S., Zhang, Z., Zhou, Q., Zhu, J. and Zhu, R.-J.: RWKV: Reinventing RNNs for the Transformer Era, *Findings of the Association for Computational Linguistics: EMNLP 2023* (Bouamor, H., Pino, J. and Bali, K., eds.), Singapore, Association for Computational Linguistics, pp. 14048–14077 (online), DOI: 10.18653/v1/2023.findings-emnlp.936 (2023).
- [25] Chaudhary, S.: Code Alpaca: An Instruction-following LLaMA Model Trained on Code Generation Instructions, <https://github.com/sahil280114/codealpaca> (2023).
- [26] LMSYS Org: FastChat: An Open Platform for Training, Serving, and Evaluating Large Language Model Based Chatbots, <https://github.com/lm-sys/FastChat> (2023).
- [27] Biderman, S., Schoelkopf, H., Anthony, Q., Bradley, H., O'Brien, K., Hallahan, E., Khan, M. A., Purohit, S., Prashanth, U. S., Raff, E., Skowron, A., Sutawika, L. and Van Der Wal, O.: Pythia: a suite for analyzing large language models across training and scaling, *Proceedings of the 40th International Conference on Machine Learning*, ICML'23, JMLR.org (2023).
- [28] Wang, Y., Ivison, H., Dasigi, P., Hessel, J., Khot, T., Chandu, K. R., Wadden, D., MacMillan, K., Smith, N. A., Beltagy, I. and Hajishirzi, H.: How far can camels go? exploring the state of instruction tuning on open resources, *Proceedings of the 37th International Conference on Neural Information Processing Systems*, NIPS '23, Red Hook, NY, USA, Curran Associates Inc. (2023).
- [29] Taori, R., Gulrajani, I., Zhang, T., Dubois, Y., Li, X., Guestrin, C., Liang, P. and Hashimoto, T. B.: Stanford Alpaca: An Instruction-following LLaMA model, https://github.com/tatsu-lab/stanford_alpaca (2023).
- [30] Clark, P., Cowhey, I., Etzioni, O., Khot, T., Sabharwal, A., Schoenick, C. and Tafjord, O.: Think you have Solved Question Answering? Try ARC, the AI2 Reasoning Challenge, arXiv:1803.05457, <https://arxiv.org/abs/1803.05457> (2018).
- [31] Welbl, J., Liu, N. F. and Gardner, M.: Crowdsourcing Multiple Choice Science Questions, *Proceedings of the 3rd Workshop on Noisy User-generated Text* (Derczynski, L., Xu, W., Ritter, A. and Baldwin, T., eds.), Copenhagen, Denmark, Association for Computational Linguistics, pp. 94–106 (online), DOI: 10.18653/v1/W17-4413 (2017).
- [32] Merity, S., Xiong, C., Bradbury, J. and Socher, R.: Pointer Sentinel Mixture Models, *International Conference on Learning Representations*, (online), available from (<https://openreview.net/forum?id=Byj72udxe>) (2017).
- [33] Smith, D., Mannering, H. and Marcu, A.: Functional Alignment Can Mislead: Examining Model Stitching, *Proceedings of the 42nd International Conference on Machine Learning* (Singh, A., Fazel, M., Hsu, D., Lacoste-Julien, S., Berkenkamp, F., Maharaj, T., Wagstaff, K. and Zhu, J., eds.), Proceedings of Machine Learning Research, Vol. 267, PMLR, pp. 55972–55998 (online), available from (<https://proceedings.mlr.press/v267/smith25a.html>) (2025).
- [34] Balogh, A. and Jelasity, M.: How Not to Stitch Representations to Measure Similarity: Task Loss Matching Versus Direct Matching, *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39, No. 15, pp. 15472–15480 (online), DOI: 10.1609/aaai.v39i15.33698 (2025).
- [35] Zhang, P. and Xin, J.: A Negative Result on Cross-Model Activation Transfer in a Pythia Multi-Hop Setting, arXiv:2606.03280, <https://arxiv.org/abs/2606.03280> (2026).